

App TaskManager: Estructura i Model de Dades

En aquest exemple complet i guiat pas a pas, construirem una aplicació **TaskManager** que integra tots els components avançats de UI estudiats. L'app permet gestionar tasques amb llistes, targetes, navegació per pestanyes i comunicació entre activitats mitjançant **Intents** i **ActivityResultLauncher**. A més, gestionem esdeveniments Android com `onSaveInstanceState`, `BroadcastReceiver` i clics.

Components i Tecnologies Usats

UI Components

- RecyclerView + CardView
- Bottom Navigation Bar
- Fragments (Home, Tasks, Profile)
- Toolbar amb menú personalitzat

Comunicació

- Intent explícit a DetailActivity
- ActivityResultLauncher per resultats
- Bundle per passar dades entre pantalles
- SharedPreferences per persistir dades

Esdeveniments Android

- `onSaveInstanceState`
- BroadcastReceiver local
- OnClickListener als ítems
- `onBackPressed` personalitzat

Estructura del Projecte

TaskManager/

- java/
 - model/ → Task.java
 - adapter/ → TaskAdapter.java
 - fragment/ → HomeFragment.java, TasksFragment.java, ProfileFragment.java
 - MainActivity.java
 - DetailActivity.java
- res/
 - layout/ → activity_main.xml, activity_detail.xml, item_task.xml, fragment_*.xml
 - menu/ → bottom_nav_menu.xml, menu_main.xml

Model de Dades: Task.java

La classe `Task` implementa **Serializable** per poder passar-la entre activitats mitjançant **Intent**.

```
// model/Task.java
public class Task implements Serializable {
    private int id;
    private String title;
    private String description;
    private boolean completed;
    private String priority; // "HIGH","MED","LOW"

    public Task(int id, String title,
                String description,
                String priority) {
        this.id = id;
        this.title = title;
        this.description = description;
        this.priority = priority;
        this.completed = false;
    }

    // Getters i Setters
    public int getId() { return id; }
    public String getTitle() { return title; }
    public boolean isCompleted() {
        return completed; }
    public void setCompleted(boolean c) {
        this.completed = c; }
    public String getPriority() {
        return priority; }
}
```

📌 Implementar **Serializable** permet enviar objectes complets via `intent.putExtra("task", task)` i recuperar-los amb `(Task) getIntent().getSerializableExtra("task")`.

TaskAdapter: RecyclerView amb CardView i Events

L'adaptador és el cor de la llista de tasques. Combina **RecyclerView.Adapter** amb l'ús de **CardView** a cada element. Definim una interfície **OnTaskClickListener** per gestionar dos tipus d'esdeveniments: clic normal per obrir el detall i clic llarg per marcar la tasca com a completada. Aquesta separació de responsabilitats fa l'adaptador reutilitzable en qualsevol pantalla.

item_task.xml (CardView)

```
<androidx.cardview.widget.CardView
    xmlns:android="..."
    xmlns:app="..."
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_margin="8dp"
    app:cardCornerRadius="8dp"
    app:cardElevation="4dp">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="16dp">

        <TextView android:id="@+id/tvTitle"
            android:textSize="16sp"
            android:textStyle="bold" />

        <TextView android:id="@+id/tvDescription"
            android:textSize="13sp"
            android:layout_marginTop="4dp" />

        <TextView android:id="@+id/tvPriority"
            android:textSize="12sp"
            android:layout_marginTop="8dp" />
    </LinearLayout>
</androidx.cardview.widget.CardView>
```

TaskAdapter.java

```
public class TaskAdapter extends
    RecyclerView.Adapter<TaskAdapter.ViewHolder> {

    private List<Task> tasks;
    private OnTaskClickListener listener;

    public interface OnTaskClickListener {
        void onTaskClick(Task task);
        void onTaskLongClick(Task task, int pos);
    }

    public static class ViewHolder
        extends RecyclerView.ViewHolder {
        TextView tvTitle, tvDescription, tvPriority;
        public ViewHolder(View v) {
            super(v);
            tvTitle = v.findViewById(R.id.tvTitle);
            tvDescription =
                v.findViewById(R.id.tvDescription);
            tvPriority =
                v.findViewById(R.id.tvPriority);
        }
    }

    @Override
    public void onBindViewHolder(
        ViewHolder holder, int position) {
        Task task = tasks.get(position);
        holder.tvTitle.setText(task.getTitle());
        holder.tvDescription.setText(
            task.getDescription());
        holder.tvPriority.setText(
            "Prioritat: " + task.getPriority());
        // Clic simple -> obrir detall
        holder.itemView.setOnClickListener(v ->
            listener.onTaskClick(task));
        // Clic llarg -> marcar completada
        holder.itemView.setOnLongClickListener(v -> {
            listener.onTaskLongClick(task, position);
            return true;
        });
    }

    public void updateTask(int pos, Task t) {
        tasks.set(pos, t);
        notifyItemChanged(pos);
    }
}
```

❑ El mètode `updateTask(pos, task)` amb `notifyItemChanged(pos)` actualitza únicament l'element canviat sense recarregar tota la llista, mantenint el rendiment òptim del RecyclerView.

DetailActivity i ActivityResultLauncher

La **DetailActivity** mostra els detalls d'una tasca i permet editar el seu títol i descripció. Quan l'usuari prem "Desar", enviem la tasca actualitzada de tornada al fragment usant `setResult(RESULT_OK, intent)`. L'**ActivityResultLauncher** és el mecanisme modern —substitut de `startActivityForResult`— per llançar activitats i rebre'n el resultat de forma segura i tipada.

TasksFragment: Registre del Launcher

```
// S'ha de registrar ABANS de onCreate
detailLauncher = registerForActivityResult(
    new ActivityResultContracts
        .StartActivityForResult(),
    result -> {
        if (result.getResultCode()
            == Activity.RESULT_OK) {
            Intent data = result.getData();
            if (data != null) {
                Task updated = (Task) data
                    .getSerializableExtra("updated_task");
                int pos = data.getIntExtra(
                    "position", -1);
                if (pos != -1 && updated != null) {
                    adapter.updateTask(pos, updated);
                }
            }
        }
    });
```

Llançament de DetailActivity

```
@Override
public void onTaskClick(Task task) {
    Intent intent = new Intent(getContext(),
        DetailActivity.class);
    intent.putExtra("task", task);
    int pos = taskList.indexOf(task);
    intent.putExtra("position", pos);
    detailLauncher.launch(intent);
}
```

DetailActivity.java — Desat i Retornar

```
private void saveAndReturn() {
    currentTask = new Task(
        currentTask.getId(),
        etTitle.getText().toString(),
        etDescription.getText().toString(),
        currentTask.getPriority());

    Intent resultIntent = new Intent();
    resultIntent.putExtra("updated_task",
        currentTask);
    resultIntent.putExtra("position",
        taskPosition);
    setResult(RESULT_OK, resultIntent);
    finish();
}

// Preserva estat en rotació de pantalla
@Override
protected void onSaveInstanceState(Bundle out) {
    super.onSaveInstanceState(out);
    out.putString("title",
        etTitle.getText().toString());
    out.putString("desc",
        etDescription.getText().toString());
}

// Restaura estat si hi ha rotació
if (savedInstanceState != null) {
    etTitle.setText(
        savedInstanceState.getString("title"));
    etDescription.setText(
        savedInstanceState.getString("desc"));
}
```

⚠ Recorda sempre cridar `super.onSaveInstanceState(out)` primer per preservar l'estat intern de les vistes del sistema abans d'afegir les teves pròpies dades al Bundle.

MainActivity: Integrant Tots els Components

La **MainActivity** és el punt d'entrada de l'app i coordina el **Toolbar**, la **Bottom Navigation Bar**, els **Fragments** i el menú de la barra d'eines. A més, implementa un **BroadcastReceiver** local per gestionar un esdeveniment personalitzat i sobrescriu `onBackPressed` per a un comportament de navegació personalitzat.

Flux Complet de l'App TaskManager

La MainActivity inicialitza Toolbar, BottomNav i els tres Fragments. Quan l'usuari navega a TasksFragment, veu la llista amb CardViews. En clicar una tasca, s'obre DetailActivity via Intent serialitzat. En desar, el resultat torna al fragment via ActivityResultLauncher. BroadcastReceiver i onSaveInstanceState garanteixen robustesa.

```
// Registra BroadcastReceiver
receiver = new TaskCompletedReceiver();
IntentFilter filter = new IntentFilter(
    "com.example.TASK_COMPLETED");
LocalBroadcastManager.getInstance(this)
    .registerReceiver(receiver, filter);

// Navegació entre fragments
private boolean onNavItemSelected(
    MenuItem item) {
    Fragment fragment;
    String title;
    int id = item.getItemId();
    if (id == R.id.nav_home) {
        fragment = new HomeFragment();
        title = "Inici";
    } else if (id == R.id.nav_tasks) {
        fragment = new TasksFragment();
        title = "Les meves Tasques";
    } else {
        fragment = new ProfileFragment();
        title = "El meu Perfil";
    }
    loadFragment(fragment, title);
    return true;
}

// Back personalitzat
@Override
public void onBackPressed() {
    if (getSupportFragmentManager()
        .getBackStackEntryCount() > 1) {
        getSupportFragmentManager().popBackStack();
    } else { super.onBackPressed(); }
}

// Desregistra per evitar memory leaks
@Override
protected void onDestroy() {
    super.onDestroy();
    LocalBroadcastManager.getInstance(this)
        .unregisterReceiver(receiver);
}
```

Components Integrats a l'App TaskManager



RecyclerView + CardView

Llista eficient de tasques amb targetes visuals i listeners per a clics simples (obrir detall) i llargs (marcar completada).



ActivityResultLauncher

Recepció tipada i segura del resultat de DetailActivity per actualitzar la llista sense recarregar tots els elements.



onSaveInstanceState

Preservació de l'estat del formulari en rotació de pantalla i recreació d'activitat, restaurant les dades via `savedInstanceState`.



Intent + Serializable

Comunicació de dades entre activitats passant objectes Task complets serialitzats via Bundle dins d'un Intent explícit.



BroadcastReceiver

Gestió d'events personalitzats de l'app amb LocalBroadcastManager. Cal desregistrar-lo a `onDestroy` per evitar leaks.



Fragment BackStack

Navegació entre fragments amb `addToBackStack` i `onBackPressed` personalitzat per controlar el flux de navegació.